MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 22 — Tests et Qualité du Code

unittest, pytest, mock — écrire du code qu'on peut prouver correct

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

20 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 22 — Tests et Qualité du Code **2**
0.1 Pourquoi tester ? 2
0.2 unittest — Le framework standard 2
0.3 Les assertions de unittest 4
0.4 pytest — Le framework moderne 5
0.5 Les mocks — Simuler les dépendances 7
0.6 Organisation des tests et couverture 8
0.7 Le TDD — Test Driven Development 9
0.8 Ce qui m'a le plus surpris dans ce chapitre 10
0.9 Résumé du Chapitre 22 10

Chapitre 22 — Tests et Qualité du Code

À retenir

Écrire du code qui fonctionne *aujourd'hui* ne suffit pas. Il faut du code qui **continue à fonctionner** quand on le modifie demain, dans six mois, en équipe. Les tests automatisés sont la réponse : ils vérifient que chaque partie du programme fait exactement ce qu'elle est censée faire. Un code sans tests est un code dont on ne peut pas être sûr. Ce chapitre couvre `unittest`, `pytest` et les mocks — les outils incontournables de la qualité logicielle en Python.

Pourquoi tester ?

Sans tests

Modifier = espérer que rien ne casse
Les bugs arrivent en production

Peur de refactorer le code
Impossible de travailler en équipe
Temps perdu en débogage manuel
Confiance nulle dans le code

Avec tests

Modifier en confiance totale
Les bugs détectés avant production
Refactoring facile et sûr

Collaboration fluide en équipe

Documentation vivante du code

Confiance absolue dans le code

unittest — Le framework standard

unittest — structure de base

```python
import unittest

# Code a tester
def calculer_moyenne(notes):
    if not notes:
        raise ValueError("La liste ne peut pas etre vide")
    return sum(notes) / len(notes)

def est_admis(note):
    return note >= 10

def formater_note(note):
    mention = ("TB" if note >= 16 else
               "B"  if note >= 14 else
```

```python
                "AB" if note >= 12 else
                "P"  if note >= 10 else "F")
    return f"{note}/20 ({mention})"


# Classe de test     herite de unittest.TestCase
class TestCalculs(unittest.TestCase):

    # setUp     execute AVANT chaque test
    def setUp(self):
        self.notes_valides  = [15, 18, 12, 9, 17]
        self.notes_parfaite = [20, 20, 20]

    # tearDown     execute APRES chaque test
    def tearDown(self):
        pass   # nettoyage si necessaire

    # Chaque methode de test commence par test_
    def test_moyenne_normale(self):
        resultat = calculer_moyenne(self.notes_valides)
        self.assertAlmostEqual(resultat, 14.2, places=1)

    def test_moyenne_parfaite(self):
        self.assertEqual(calculer_moyenne(self.notes_parfaite),
            20)

    def test_moyenne_liste_vide(self):
        with self.assertRaises(ValueError):
            calculer_moyenne([])

    def test_admis_vrai(self):
        self.assertTrue(est_admis(10))
        self.assertTrue(est_admis(15))
        self.assertTrue(est_admis(20))

    def test_admis_faux(self):
        self.assertFalse(est_admis(9))
        self.assertFalse(est_admis(0))

    def test_formater_mention_tb(self):
        self.assertIn("TB", formater_note(18))

    def test_formater_mention_f(self):
        self.assertIn("F", formater_note(5))


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Les assertions de unittest

Toutes les assertions importantes

```python
import unittest

class TestAssertions(unittest.TestCase):

    def test_egalite(self):
        self.assertEqual(2 + 2, 4)
        self.assertNotEqual(2 + 2, 5)

    def test_booleens(self):
        self.assertTrue(10 > 5)
        self.assertFalse(5 > 10)

    def test_none(self):
        self.assertIsNone(None)
        self.assertIsNotNone(42)

    def test_appartenance(self):
        self.assertIn("Python", ["Java", "Python", "C++"])
        self.assertNotIn("Rust", ["Java", "Python", "C++"])

    def test_types(self):
        self.assertIsInstance(42,       int)
        self.assertIsInstance("hello", str)
        self.assertIsInstance([1,2,3], list)

    def test_flottants(self):
        # Ne jamais assertEqual sur des flottants !
        self.assertAlmostEqual(0.1 + 0.2, 0.3, places=10)

    def test_collections(self):
        self.assertListEqual([1, 2, 3], [1, 2, 3])
        self.assertDictEqual({"a": 1}, {"a": 1})
        self.assertSetEqual({1, 2, 3}, {3, 1, 2})

    def test_exceptions(self):
        with self.assertRaises(ZeroDivisionError):
            result = 1 / 0

        with self.assertRaises(ValueError) as ctx:
            int("abc")
        self.assertIn("invalid literal", str(ctx.exception))

    def test_comparaisons(self):
        self.assertGreater(5, 3)
        self.assertLess(3, 5)
        self.assertGreaterEqual(5, 5)
```

```
        self.assertLessEqual(3, 3)
```

pytest — Le framework moderne

pytest — simplicité et puissance

```python
# pytest s'installe avec : pip install pytest
# On lance avec : pytest nom_fichier.py -v

# FICHIER : test_calculatrice.py
# Pas besoin de classe ni d'import unittest !

def additionner(a, b):    return a + b
def soustraire(a, b):     return a - b
def multiplier(a, b):     return a * b
def diviser(a, b):
    if b == 0:
        raise ZeroDivisionError("Division par zero")
    return a / b


# Tests simples     juste des fonctions avec assert
def test_addition_positive():
    assert additionner(3, 4) == 7

def test_addition_negative():
    assert additionner(-2, -3) == -5

def test_addition_zero():
    assert additionner(0, 5) == 5

def test_soustraction():
    assert soustraire(10, 4) == 6

def test_multiplication():
    assert multiplier(6, 7) == 42

def test_division_normale():
    assert diviser(10, 2) == 5.0

def test_division_par_zero():
    import pytest
    with pytest.raises(ZeroDivisionError):
        diviser(10, 0)

def test_division_flottant():
    import pytest
    assert diviser(1, 3) == pytest.approx(0.333, rel=1e-2)
```

pytest — fixtures et paramétrisation

```python
import pytest

# fixture     ressource partagee entre tests
@pytest.fixture
def liste_notes():
    return [15, 18, 12, 9, 17, 14]

@pytest.fixture
def etudiant():
    return {"prenom": "Alice", "note": 15, "ville": "Paris"}

def test_longueur_notes(liste_notes):
    assert len(liste_notes) == 6

def test_note_max(liste_notes):
    assert max(liste_notes) == 18

def test_etudiant_admis(etudiant):
    assert etudiant["note"] >= 10

# parametrize     tester plusieurs cas d'un coup
@pytest.mark.parametrize("note, attendu", [
    (20, "TB"),
    (16, "TB"),
    (15, "B"),
    (14, "B"),
    (12, "AB"),
    (10, "P"),
    (9,  "F"),
    (0,  "F"),
])
def test_mention(note, attendu):
    def mention(n):
        return ("TB" if n >= 16 else
                "B"  if n >= 14 else
                "AB" if n >= 12 else
                "P"  if n >= 10 else "F")
    assert mention(note) == attendu

# Marquer des tests a sauter
@pytest.mark.skip(reason="Fonctionnalite pas encore implementee
    ")
def test_future():
    assert False

@pytest.mark.xfail(reason="Comportement connu")
def test_connu_qui_echoue():
    assert 1 == 2
```

Les mocks — Simuler les dépendances

unittest.mock — isoler le code testé

```python
from unittest.mock import Mock, patch, MagicMock
import unittest

# Simuler un objet avec Mock
mock_api = Mock()
mock_api.get_data.return_value = {"status": "OK", "data":
    [1,2,3]}
mock_api.get_user.return_value = {"nom": "Alice", "age": 25}

# Utiliser le mock
resultat = mock_api.get_data()
print(resultat)   # {"status": "OK", "data": [1,2,3]}

# Verifier les appels
mock_api.get_data.assert_called_once()
mock_api.get_user("alice@gmail.com")
mock_api.get_user.assert_called_with("alice@gmail.com")


# Exemple realiste : tester sans connexion reseau
def recuperer_taux_change(devise):
    """Appelle une API externe     ne pas tester avec vraie API
        ."""
    import urllib.request
    url = f"https://api.rates.io/{devise}"
    # ... appel reseau reel

def calculer_prix_en_euros(prix_usd, taux):
    return prix_usd * taux

class TestPrix(unittest.TestCase):

    @patch("builtins.input", return_value="42")
    def test_avec_input_simule(self, mock_input):
        valeur = int(input("Entrez un nombre : "))
        self.assertEqual(valeur, 42)

    def test_avec_mock_taux(self):
        # On teste la logique SANS appel reseau
        taux_simule = 0.92
        prix_usd    = 100
        resultat    = calculer_prix_en_euros(prix_usd,
            taux_simule)
        self.assertAlmostEqual(resultat, 92.0)

    @patch("builtins.open",
```

```
            unittest.mock.mock_open(read_data="Alice,15\nBob,18\n"))
    def test_lecture_fichier_simule(self, mock_file):
        with open("notes.csv") as f:
            contenu = f.read()
        self.assertIn("Alice", contenu)
        self.assertIn("Bob", contenu)
```

Organisation des tests et couverture

Structure professionnelle d'un projet testé

```
% Structure recommandee :
%
% mon_projet/
%   src/
%     mon_projet/
%       __init__.py
%       calculatrice.py
%       etudiant.py
%   tests/
%       __init__.py
%       test_calculatrice.py
%       test_etudiant.py
%       conftest.py           fixtures partagees pytest
%   pytest.ini                configuration pytest
%   requirements.txt

% Contenu de pytest.ini
% [pytest]
% testpaths = tests
% python_files = test_*.py
% python_classes = Test*
% python_functions = test_*
% addopts = -v --tb=short

% Lancer les tests :
% pytest                            tous les tests
% pytest tests/test_calcul.py       un fichier
% pytest -k "moyenne"               tests contenant "moyenne"
% pytest -x                         arreter au premier echec

% Couverture de code avec pytest-cov :
% pip install pytest-cov
% pytest --cov=src --cov-report=html
%     Ouvre htmlcov/index.html dans le navigateur

% conftest.py     fixtures globales
```

```python
import pytest

@pytest.fixture(scope="session")
def base_de_donnees():
    """Connexion BDD partagee pour toute la session."""
    print("\nConnexion BDD...")
    db = {"etudiants": [], "cours": []}
    yield db
    print("\nFermeture BDD...")

@pytest.fixture(scope="function")
def etudiant_alice():
    return {"prenom": "Alice", "note": 15}
```

Le TDD — Test Driven Development

TDD — écrire les tests AVANT le code

```python
import pytest

# ETAPE 1 : Ecrire le test AVANT la fonction
# Le test echoue d'abord     c'est normal !

def test_valider_email_valide():
    assert valider_email("alice@gmail.com") == True

def test_valider_email_sans_at():
    assert valider_email("alicegmail.com") == False

def test_valider_email_sans_domaine():
    assert valider_email("alice@") == False

def test_valider_email_vide():
    assert valider_email("") == False

# ETAPE 2 : Ecrire le MINIMUM de code pour passer
import re

def valider_email(email):
    if not email:
        return False
    pattern = r"^[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}$"
    return bool(re.match(pattern, email))

# ETAPE 3 : Refactorer en confiance
# Les tests garantissent qu'on ne casse rien

# CYCLE TDD : Rouge     Vert     Refactor
```

```
# Rouge  : le test echoue (code absent)
# Vert   : le test passe (code minimal)
# Refactor : ameliorer sans casser les tests
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. Les tests sont une forme de documentation vivante. Un test bien écrit dit exactement ce que le code est censé faire — avec des exemples concrets. `test_moyenne_liste_vide` documente que la fonction lève une `ValueError` pour une liste vide. C'est plus précis et plus fiable que n'importe quel commentaire, parce qu'il est **exécutable et vérifiable**. Les tests ne remplacent pas la documentation — ils en sont la forme la plus honnête.

2. `pytest.mark.parametrize` — tester cent cas en cinq lignes. Avant de connaître `parametrize`, je dupliquais mes tests pour chaque cas. Avec `parametrize`, je déclare tous les couples (entrée, résultat attendu) en un tableau et pytest génère automatiquement un test pour chacun. Un seul test parametrisé peut couvrir des dizaines de cas limites sans aucune duplication de code.

3. Les mocks révèlent la qualité de l'architecture. Si mocker un objet est difficile, c'est souvent signe que le code est trop couplé. Une fonction qui appelle directement une API externe est difficile à tester — et difficile à maintenir. Apprendre à mocker m'a appris à **concevoir des fonctions testables** : petites, avec des dépendances injectables. Les tests améliorent l'architecture autant qu'ils la vérifient.

Résumé du Chapitre 22

Ce que j'ai appris dans ce chapitre :

- **unittest** — framework standard : `TestCase`, `setUp`, assertions.
- `assertEqual`, `assertTrue`, `assertRaises` — les assertions fondamentales.
- **pytest** — tests simples avec `assert`, sans boilerplate.
- `@pytest.fixture` — ressources partagées entre tests.
- `@pytest.mark.parametrize` — tester N cas en une déclaration.
- **Mock** — simuler des dépendances externes sans appels réels.
- `@patch` — remplacer temporairement une fonction ou un module.
- **TDD** — écrire les tests avant le code : Rouge → Vert → Refactor.
- `pytest-cov` — mesurer la couverture de code par les tests.

"Le code non testé est du code cassé par définition. Un test qui passe n'est pas une victoire — c'est une garantie."